Study Proof

A learning-evidence layer for the Study Tracker desktop app | TeachAthon 2026 submission draft

Core idea: after a study session, require a student explanation before AI help; give feedback tied to the student's chosen note; then ask for an unaided delayed retry.

The problem

Study Tracker already records sessions, minutes, XP, medals, streaks, a calendar and notes. These show effort, but a long study session can still end in a false sense of understanding. The TeachAthon opening slides identify this “hallucinated understanding” alongside skills erosion, assessment integrity and the need to make responsible AI use concrete. A general chatbot can give fluent answers while hiding what the learner can do unaided. Our question is: can a tracker make learning effort and learning evidence visible together?

The proposed experience

Step	Student and AI action	Visible evidence
1. Choose	After a logged session, select one existing note and a concept.	The app records which material was used.
2. Explain	Answer a short question without AI hints; rate confidence.	An immutable first attempt separates student thinking from model help.
3. Challenge	AI asks one follow-up and offers a hint tied to an exact note passage.	The student can inspect the quoted source and flag an error.
4. Retry	Revise the explanation, then repeat an unaided transfer question days later.	The app shows both attempts next to time studied; no AI score is treated as proof of mastery.

Why this differs from another study chatbot

The official 2025 TeachAthon archive already includes course-grounded Socratic tutoring, AI quizzes, and gamified planning. Study Proof puts the student's first and later unaided attempts at the centre and uses the model only to prompt and support reflection. It builds on the existing tracker rather than requiring a new learning platform. Its potential users are students who study from personal notes, especially when they need a quick check before assuming a topic is understood.

How it would work

A narrow, buildable prototype with explicit limits

Architecture and scope

The existing Python/CustomTkinter app already stores notes and sessions locally with autosave and backups. Add one “Check understanding” action, a versioned practice-attempt record, a simple delayed-review scheduler, and a model adapter. The adapter receives only the chosen note excerpt and the student's first answer. It returns a question, one hint, a source passage, and an “unsupported” state. Before display, the app checks that the cited passage actually occurs in the selected note. When grounding fails, it says the note does not support the claim. The AI call runs outside the GUI thread; the tracker and manual practice flow still work without it.

Demonstration plan

Use a small, team-written computer-science note about a binary-search invariant. Show a student's incomplete first explanation, a model follow-up linked to a precise passage, the student's revision, and a later unaided question scheduled in the calendar. Show the original answer and note passage side by side. A prerecorded response must be labeled as a mock; a functioning model call must be shown as such. This sequence demonstrates the educational mechanism as well as the software.

Evidence and evaluation

Retrieval-practice experiments found better delayed retention from testing than repeated study in the studied conditions [1]. Recent AI-tutoring studies suggest that pedagogy and grounding can matter [2, 3], while a randomized university trial found that tutor access could displace existing study activity and lower grades in same-course comparisons [4]. These findings motivate a cautious trial; they do not establish that Study Proof improves learning. In a pilot, compare Study Proof with ordinary tracking on the same topic. Use an unaided baseline and a new unaided transfer question several days later, scored by blinded humans. Also measure citation errors, student corrections, time cost, access barriers, and feedback usefulness. Engagement and XP alone are not success measures.

Near-term delivery path

By	Concrete output
19 Oct	Register team; finalize the one-flow design and test note.
22 Oct	Working note-to-feedback slice with source validation and a manual fallback.
24 Oct	Usability and safety checks; fix misleading feedback and blocking errors.
26 Oct	Submit three-page report and clearly labeled live demo or video.

Prototype success means a usable, honest learning-check loop. Claims about improved retention require a later controlled study.

Responsible deployment and judging fit

Student agency, traceable feedback, and a credible completion path

Appropriate use and risk controls

Risk	Design response
Skill erosion / cheating	First attempt is always unaided. The tool is for practice, not assessed-answer generation; it reminds students to check course rules and disclose AI use when required.
Hallucinated feedback	Feedback is scoped to a selected note and points to a checked passage. If unsupported, the app abstains and asks the student to verify.
Privacy / unequal access	The existing local tracker remains usable without AI. AI is opt-in; only selected material is shared with an approved provider, or a local model is used if hardware allows. No background upload or instructor surveillance dashboard.
Misleading “mastery” claims	The display labels attempts and confidence; any AI judgement is editable feedback. A delayed unaided task and human-scored pilot supply stronger evidence.

Edinburgh's student guidance requires course-level rule checking and cautions against unacknowledged AI-generated assessed content and over-reliance [5]. UNESCO emphasizes privacy and human agency [6]. A university ELM connection is a possible future route, but programmatic access for this app has not been verified. The team would confirm permissions and data handling before any real student pilot.

Alignment with the four TeachAthon criteria

Criterion	What the submission provides
Scope & impact	A concrete gap between logged study time and demonstrated understanding, with a workflow students can use after every session.
Feasibility	An existing desktop app, narrow data/model additions, source checking, staged build plan, and offline fallback.
Ethics & use	Unaided-first practice, course-policy awareness, consent, minimised data sharing, abstention and user correction.
Demo & completion	A focused end-to-end scenario plus a pilot that tests delayed unaided learning rather than clicks or XP.

Selected sources

[1] Roediger & Karpicke (2006), Test-Enhanced Learning. [2] Yang et al. (2026 preprint), When AI Tutors Speak. [3] College science AI tutor RCT. [4] Randomized university trial (2026 working paper). [5] University of Edinburgh student guidance, rev. Mar 2026. [6] UNESCO GenAI guidance. Event criteria: TeachAthon 2026 opening slides, pp. 5, 7-9. Product basis: supplied Study Tracker archive. Prior art: 2025 official archive.